Санкт-Петербургский государственный университет

Прикладная математика, программирование и искусственный интеллект

Отчет

По учебной практике 4

Использование комплексных нейронных сетей при магнито-резонансной томографии в медицине

Выполнила:

Балта Валерия Евгеньевна
группа: 24.Б23

Научный руководиетль:

Райтманн Фолькер
Профессор кафедры
прикладной кибернетики

Санкт-Петербург
2026

Содержание

1 Введение **2**

2 Цель работы **2**

3 Постановка задач **3**

4 Теоретические основы MRF и комплексно-значных нейронных сетей **4**

4.1 Основаная проблема МРТ 4
4.2 Физические параметры: T1, T2 и B0 4
4.3 Математическая модель: уравнения Блоха и физика МРТ-сигнала 5
4.4 MRF 7
4.5 Традиционное решение обратной задачи 7
4.6 Преимущества глубокого обучения в MRF 7
4.7 Использование комплексно-значной нейросети 8
4.8 Обучение комплексно-значных нейронных сетей: исчисление Виртингера 9

5 Программная реализация **11**

5.1 Формирование пространства параметров 11
5.2 Програмный симулятор Блоха 11
5.3 Архитектура нейронной сети 14
5.4 Модуль обучения и оптимизации 15
5.5 Оценка качества и визуализация результатов 16

6 Выводы **18**

7 Заключение **18**

8 Список лиетратуры **19**

9 Приложение **20**

1 Введение

Магнитно-резонансная томография(МРТ) является одним из наиболее эффективных, информативных и безопасных методов медицинской диагностики, позволяющих получать изображение внутренних тканей человека.Однако традиционные MRI-методы требуют нескольких отдельных сканирований.

MRI-фингерпринтинг (Magnetic Resonance Fingerprinting, MRF) позволяет за одно сканирование одновременно получать количественные значения параметров тканей: T1, T2, плотность протонов, а также параметров магнитного поля B0 и B1. Суть метода заключается в том, что каждая ткань под действием специальной импульсной последовательности выдаёт уникальный комплекснозначный сигнал. Задача сводится к восстановлению параметров тканей по этому сигналу (обратное отображение).

Преобладающий подход основан на словаре, где тестовый сигнал MRI сравнивается с уже извесными параметрами тканей. Но этот метод довольно медленный, когда велико промстранство тканей.

В статье «Better Than Real: Complex-Valued Neural Nets for MRI Fingerprinting» предложено использовать комплексно-значные нейронные сети с новой функцией активации - комплексной кардиоидой. Эта активация чувствительна к фазе сигнала и позволяет сети работать с комплексными данными более эффективно.

Данная работа посвящена изучению и программной реализации именно такого подхода: комплексно-значной нейронной сети с кардиоидной активацией для задачи MRI-фингерпринтинга.

2 Цель работы

Исследование применения комплекснозначных нейронных сетей в методе Magnetic Resonance Fingerprinting для поиска обратнгого отображения, а также разработка програмной реализации сети и экспериментальное сравнение её точности и скорости с традиционными методами.

3 Постановка задач

- Изучить принципы работы магнитно-резонансной томографии и метода MRI fingerprinting.
- Реализовать комплексно-значную нейронную сеть с кардиоидной функцией активации.
- Оценить точность работы программы.
- Провести сравнительный анализ эффективности различных методов реконструкции параметров тканей.

4 Теоретические основы MRF и комплексно-значных нейронных сетей

4.1 Основаная проблема МРТ

Магнитно-резонансная томография (МРТ) является одним из наиболее важных методов медицинской диагностики. Она позволяет получать изображения внутренних органов без использования ионизирующего излучения. Основой МРТ является поведение атомов водорода в магнитном поле.

В классической МРТ получают изображения, в которых различия между тканями видны визуально (например, одна ткань светлее, другая темнее). Однако такие изображения являются не колличественными а качественными. Это означает, что врач видит разницу между такнями, но не получает численные их параметры.

Для решения этой проблемы был предложен метод MRI fingerprinting (MRF). Его идея заключается в том, чтобы не просто получить изображение, а определить физические параметры ткани, такие как T1, T2 и B0, которые позволяют более точно выявлять патологии.

4.2 Физические параметры: T1, T2 и B0

Параметр T1

Параметр T1 описывает процесс, называемый продольной релаксацией. После воздействия радиочастотного импульса протоны водорода отклоняются от своего равновесного состояния. Затем они начинают возвращаться обратно.

T1 показывает, насколько быстро происходит это восстановление. Чем больше значение T1, тем медленнее происходит возвращение в исходное состояние.

Разные ткани имеют разные значения T1. Благодаря этому можно различать ткани на изображении.

Параметр T2

Параметр T2 связан с поперечной релаксацией. Он описывает, как быстро протоны теряют синхронность своего движения. Чем меньше T2, тем быстрее сигнал затухает.

Параметр B0

Параметр B0 — это основное магнитное поле томографа. В идеале оно должно быть одинаковым во всей области, но на практике существуют небольшие отклонения. У него существуют небольшие отклонения, которые влияют на протоны и на сигнал, соответс-

венно. Поэтому B0 тоже важный параметр, который необходимо учитывать.

4.3 Математическая модель: уравнения Блоха и физика МРТ-сигнала

Чтобы понять, как нейросеть восстанавливает параметры ткани, необходимо проследить путь формирования сигнала от отдельного атома до цифрового «отпечатка». В основе этого процесса лежат протоны водорода, которые ведут себя как крошечные магнитные стрелки. В обычном состоянии они направлены хаотично, но как только человек оказывается в сильном магнитном поле томографа B_0, эти «стрелки» выстраиваются вдоль поля. В результате в теле пациента возникает суммарный вектор макроскопической намагниченности $\vec{M}$, который изначально направлен строго вдоль оси Z.

Движение этого вектора описывается уравнениями Блоха. Если представить вектор $\vec{M}$ в виде физического объекта, то магнитное поле действует на него как крутящий момент. Это взаимодействие описывается уравнением:

$$\frac{d\vec{L}}{dt} = \vec{\mu} \times \vec{B}$$

Это заставляет вектор не просто поворачиваться, а вращаться вокруг оси поля. Скорость этого вращения называется частотой Ларморa и напрямую зависит от силы поля:

$$\omega = \gamma B_0$$

. Пока вектор просто вращается вокруг оси Z, мы не можем его зафиксировать, поэтому чтобы получить сигнал, нам нужно отклонить его в поперечную плоскость XY с помощью радиочастотного импульса под определенным углом α.

Как только импульс прекращается, начинаются два параллельных процесса, которые и создают уникальность МРТ-изображений: продольная и поперечная релаксация. Продольная релаксация, характеризуемая временем T_1, — это процесс возвращения в исходное состояние. Вектор намагниченности стремится снова выстроиться вдоль оси Z, отдавая накопленную энергию окружающим тканям. Математически это описывается дифференциальным уравнением

$$\frac{dM_z}{dt} = (M_0 - M_z)/T_1$$

. Его решение показывает, что намагниченность восстанавливается постепенно:

$$M_z(t) = M_z(0)e^{-t/T_1} + M_0(1 - e^{-t/T_1})$$

. Разные ткани имеют разное время T_1, что позволяет нам видеть границы между ними.

Одновременно с этим происходит поперечная релаксация T_2, которая отвечает за «затухание» сигнала в плоскости XY. В момент импульса все протоны вращаются синхронно, создавая максимальный сигнал и взаимодействуя друг с другом. Их фазы рассогласовываются, и суммарный сигнал в плоскости XY исчезает. В комплексном представлении, которое используется в программной реализации симулятора, этот процесс выглядит как движение вектора по спирали к центру:

$$M_{xy}(t) = M_{xy}(0)e^{-t/T_2}$$

. При этом фаза (угол поворота) вектора дополнительно искажается из-за неоднородностей магнитного поля B_0. Таким образом, регистрируемый комплексный сигнал в каждый момент времени — это результат измерения скорости восстановления по Z и скорости затухания в плоскости XY.

В технологии магнитно-резонансного фингерпринтинга мы не ждем, пока система полностью вернется в покой. Мы подаем серию импульсов очень быстро, один за другим. Из-за этого вектор намагниченности приходит в состояние так называемого динамического равновесия. Стационарное состояние M_{ss} достигается, когда количество восстановившейся намагниченности за период TR в точности равно количеству, "выбитому"импульсом. Из рекуррентного соотношения $M_z(t_{n+1}) = M_z(t_n)\cos(\alpha)e^{-TR/T_1} + M_0(1 - e^{-TR/T_1})$ Олман выводит итоговую формулу для сигнала перед импульсом:

$$M_{ss} = \frac{M_0(1 - e^{-TR/T_1})}{1 - \cos(\alpha)e^{-TR/T_1}}$$

Одной из ключевых точек оптимизации здесь является угол Эрнста - угол отклонения, который максимизирует доступную поперечную намагниченность $M_\perp = M_{ss}\sin(\alpha)$ для конкретного типа ткани. Он находится через производную функции сигнала и выражается формулой:

$$\alpha_{Ernst} = \arccos(e^{-TR/T_1})$$

Численное решение уравнений Блоха для такой сложной последовательности позволяет получить временной ряд — «отпечаток

пальца» ткани. Каждая точка этого ряда содержит информацию о T_1, T_2 и B_0, а задача нейросети Complex Cardioid заключается в том, чтобы разгадать эту зависимость и вернуть точные физические значения параметров для каждого пикселя изображения.

4.4 MRF

Метод MRI fingerprinting основан на идее, что каждая ткань имеет уникальный «отпечаток» сигнала.

Во время сканирования используется последовательность импульсов с разными параметрами, в ответ на которые такини генерируют сигналы, изменяющиеся во времени.

Для каждой комбинации параметров генерируется временной вектор ожидаемого сигнала, который и называется «fingerprint» - отпечатк. Итоговый словарь может содержать сотни тысяч или даже миллионы записей.

Обратная задача MRF заключается в том, чтобы по сигналу определить параметры (T1, T2, B0).

4.5 Традиционное решение обратной задачи

Традиционно для решения обратной задачи используется словарный поиск. Для каждой комбинации параметров генерируется временной вектор ожидаемого сигнала. Итоговый словарь может содержать сотни тысяч или даже миллионы записей. Каждая временная последовательность из сканера сравнивается со всеми последовательностями из предварительно симулированного словаря.

Основным ограничением словарного подхода является его вычислительная стоимость, которая растет экспоненциально с добавлением новых параметров.

4.6 Преимущества глубокого обучения в MRF

Использование нейронных сетей для реконструкции MRF дает ряд преимуществ:

- Эффективность памяти: веса нейронной сети занимают существенно меньше места, чем словарь.
- Обученная сеть способна лучше обобщать данные и игнорировать шум по сравнению с простым поиском ближайшего соседа.

- Время обработки не зависит от того, насколько детальным был обучающий набор данных.

Однако, вещественные нейронные сети не подходят для обработки комплекснозначного MR-сигнала . Они разбивают комплексный сигнал на два независимых числа, из-за чего теряют математическую связь между ними.

4.7 Использование комплексно-значной нейросети

Комплекснозначные нейронные сети - это архитектуры, в которых входные данные, веса, смещения и функции активации определены в поле комплексных чисел $\mathbb{C}$. Они естественным образом подходят для задач обработки сигналов, где амплитуда и фаза неразрывно связаны.

Обычные функции, такие как ReLU($max(0, x)$) или sigmoid, не подходят напрямую в качестве активационных функций для комплексных нейронных сетей, так как они определены только для вещественных чисел.

Исторические подходы

До появления специализированных функций исследователи использовали следующие варианты :

- Применение ReLU отдельно к реальной и мнимой частям: $f(z) = ReLU(Re(z)) + i \cdot ReLU(Im(z))$. Это искажает фазу сигнала и разрушает векторную природу числа.

- Функция Siglog, которая нелинейно сжимает магнитуду, сохраняя фазу: $f(z) = \frac{z}{1+|z|}$. Она берет любое комплексное число и просто делает его чуть короче (сжимает амплитуду), но никогда не превращает его в чистый ноль (разве что само z изначально было нулем).

Комплексная кардиоида

В рассматриваемой работе в качетсве акттивационной функции предложена комплексная кардиоида (complex cardioid).

$$f(z) = \frac{1}{2}(1 + cos(z))z$$

Где $\angle z$ — это фаза (аргумент) входного комплексного числа.

Эта функция обладает свойствами:

- Сохраняет фазу входного сигнала.
- Для положительных вещественных чисел коэффициент равен 1 (сохраняется). Для отрицательных вещественных чисел коэффициент равен 0 (блокировка сигнала).
- Графически снижает амплитуду комплексного числа в зависимости от его фазы.
- Если вход ограничен только вещественными числами, функция в точности превращается в стандартный ReLU.

Приведенное ниже изображение наглядно показывает как работает комплексная кардиоида.

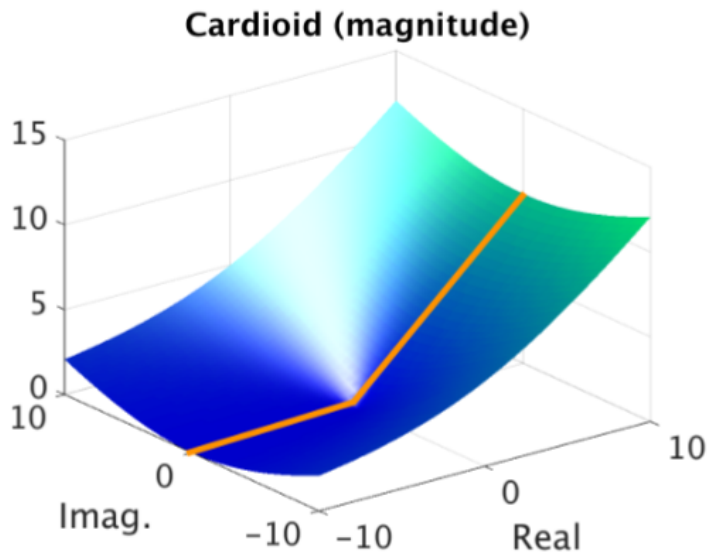


Рис. 1: Комплексная кардиоида

Нижняя плоскость - это входное комплексное число z. Вертикальная ось - величина числа на выходе после функции активации. Оранжевая линия — это связь с обычным ReLU. Она проходит строго по вещественной оси.

4.8 Обучение комплексно-значных нейронных сетей: исчисление Виртингера

Обучение нейронных сетей в комплексном пространстве требует применения специализированного математического аппарата, существенно отличающегося от классических вещественных моделей.

В стандартном алгоритме обратного распространения ошибки целью является минимизация функции потерь L, которая оценивает расхождение между предсказанием сети и истинным значением. Особенность задачи заключается в том, что функция потерь всегда возвращает вещественное число, осуществляя отображение из комплексного пространства параметров в вещественное скалярное поле: $\mathbb{C}^n \to \mathbb{R}$, но такая функция потерь не удовлетворяет условиям Коши-Римана, а следовательно, не является голоморфной. Из этого следует невозможность применения стандартного комплексного дифференцирования для поиска оптимальности. Элегантным и математически строгим решением проблемы дифференцируемости неаналитических функций является использование исчисления Виртингера.

Основная идея исчисления Виртингера состоит в том, чтобы рассматривать любую неаналитическую функцию комплексного переменного $f(z)$, как функцию двух формально независимых переменных: самого числа z и его комплексно-сопряженной копии $\bar{z} = x - iy$. Согласно данному подходу, производные Виртингера определяются через линейные комбинации классических частных производных по вещественной и мнимой частям:

$$\frac{\partial f}{\partial z} = \frac{1}{2}\left(\frac{\partial f}{\partial x} - i\frac{\partial f}{\partial y}\right), \quad \frac{\partial f}{\partial \bar{z}} = \frac{1}{2}\left(\frac{\partial f}{\partial x} + i\frac{\partial f}{\partial y}\right)$$

При обучения нейронной сети функция потерь $L(w, \bar{w})$ зависит от матрицы комплексных весов w. Основная задача оптимизатора — определить направление наискорейшего убывания функции в многомерном комплексном пространстве. Для вещественнозначной функции от комплексных переменных направлением антиградиента является не стандартная производная по весу $\nabla_w L$, а производная по сопряженному весу — комплексный коградиент $\nabla_{\bar{w}} L$.

Исходя из этого, правило обновления весовых коэффициентов при использовании алгоритма комплексного стохастического градиентного спуска принимает следующий вид:

$$w_{n+1} = w_n - \alpha \nabla_{\bar{w}} L$$

где α — гиперпараметр скорости обучения, а $\nabla_{\bar{w}} L$ представляет собой вектор-столбец частных производных Виртингера по сопряженным весам на текущей итерации n.

Использование именно сопряженного градиента является критически важным для обработки МР-фингерпринтинга. Такой подход гарантирует, что при корректировке весов сеть будет корректно оптимизировать параметры комплексного домена, одновременно учитывая затухание сигнала T_2, и влияние неоднородностей B_0.

5 Программная реализация

5.1 Формирование пространства параметров

Геренируем значения для параметров T1, T2, B0 в физиологически обоснованных диапазонах от 100 до 5000 мс для T1, от 20 до 2000мс для T2 и от -250 до 250 Гц для B0. Для обучения генерируется массив из 100 000 независимых комбинаций параметров, что является оптимальнам значением.

```
# Константы
N_SAMPLES = 100000
N_PULSES = 500
T1 = np.random.uniform(100, 5000, N_SAMPLES)
T2 = np.random.uniform(20, 2000, N_SAMPLES)
B0 = np.random.uniform(-250, 250, N_SAMPLES)
```

5.2 Програмный симулятор Блоха

Программный симулятор Блоха выступает в роли функции, преобразующей тройку физических чисел в комплексный временной ряд. Моделирование проводится дискретно для последовательности из 500 импульсов.

Для этого создаются два списка констант (они будут одинаковыми для всех 100 000 примеров):

1. Список углов наклона (FA): Значения, меняющиеся по синусоиде от 0° до 60°. Это сила, с которой мы «бьем» по вектору намагниченности радиоимпульсом, отклоняя его от вертикали. Если мы будем бить всегда с одинаковой силой, сигнал от разных тканей быстро станет очень похожим. Когда мы плавно меняем угол, то заставляем каждую ткань реагировать уникально. В итоге получается сложная кривая, по которой нейросеть сможет их различить.

2. Список времен повторения (TR): Случайные числа от 10 до 15 мс. Это пауза между двумя ударами импульса. Случайный TR не дает системе прийти в стабильное состояние. Это позволяет сделать отклик ткани максимально непохожим на другие.

```
# Списки констант для всех примеров
FA = np.sin(np.linspace(0, np.pi, N_PULSES)) * (60 * np.pi / 180) # Сила с которой бьем по вектору намагниченности
TR = np.random.uniform(10, 15, N_PULSES) #Пауза между ударами
```

Теперь для каждой нашей тройки (например, $T1 = 1000, T2 = 80, B_0 = 5$) программа запускает физический цикл на 500 шагов. Внутри каждого шага (от 1 до 500):

- Удар импульсом: Программа берет текущий угол из списка FA и «наклоняет» виртуальный вектор намагниченности (матрица поворота).
- Ожидание (TR): Пока идет время до следующего импульса, вектор «расслабляется» по формулам экспонент $e^{-TR/T1}$ и $e^{-TR/T2}$.
- Поворот (B_0): Вектор дополнительно прокручивается по кругу на угол, зависящий от ошибки поля B_0.
- Снятие показаний: Мы берем координаты вектора в горизонтальной плоскости (M_x и M_y) и записываем их как комплексное число $M_x + iM_y$.
- Выход модуля: Матрица комплексных чисел размером $100\,000 \times 500$, где каждая строка - это уникальный «отпечаток пальца» ткани.

```
 # Вектор намагниченности: M_x=0, M_y=0, M_z=1
 M = np.zeros((3, n_samples)) #Матрица 100000 векторов намагниченности по осям X Y Z
 M[2, :] = 1.0 #Все векторы изачально смотрят вверх вдоль(OZ)

 signals = np.zeros((n_samples, n_pulses), dtype=np.complex128) #Таблица для отпечатков сигналов

 for i in range(n_pulses):
     alpha = FA[i] #Текущий угол удара
     tr = TR[i] # Текущее время паузы

     # 1. Удар импульсом бьем( по вектору волной). вычисляем координаты после поворота на угол в плоскости YZ
     M_new_y = M[1]*np.cos(alpha) + M[2]*np.sin(alpha)
     M_new_z = -M[1]*np.sin(alpha) + M[2]*np.cos(alpha)
     M[1] = M_new_y
```

```
        M[2] = M_new_z

        # 2. Снятие показанийкоордината( X становится реальной частью, а Y — мнимой)
        signals[:, i] = M[0] + 1j * M[1]

        # 3. Затухание (T2) и восстановление (T1)
        E1 = np.exp(-tr / T1)
        E2 = np.exp(-tr / T2)
        theta = 2 * np.pi * B0 * (tr / 1000.0) #Вращение вектора вокруг Z из за неидеальности магнитного поляесть( B0)

        #Обновление координат вектора
        M_xy = (M[0] + 1j * M[1]) * E2 * np.exp(-1j * theta) #комплексное число становится короче и поворачивается
        M[0] = np.real(M_xy)
        M[1] = np.imag(M_xy)
        M[2] = M[2] * E1 + (1 - E1)
```

Перед подачей в нейросеть сигналы из симулятора проходят два этапа обработки:

1. Нормализация: Каждый сигнал нужно разделить на его длину (норму), чтобы сеть не зависела от общей яркости картинки. Значения $T1$ и $T2$ нужно привести к диапазону $[0, 1]$, иначе градиенты будут слишком большими.

2. Добавление шума: К комплексным отсчетам добавляется гауссовский шум с уровнем pSNR=40. Это необходимо для обеспечения устойчивости программы при работе с реальными, зашумленными сигналами томографа.

```
def add_complex_noise(signal, psnr=40):
    # Добавление гауссовского шума
    signal_power = np.mean(np.abs(signal)**2)
    noise_power = signal_power / (10**(psnr/10))
    noise_real = np.random.normal(0, np.sqrt(noise_power/2), signal.shape)
    noise_imag = np.random.normal(0, np.sqrt(noise_power/2), signal.shape)
    return signal + (noise_real + 1j * noise_imag)

# 1. Нормализация комплексных сигналов
norms = np.linalg.norm(X_complex, axis=1, keepdims=True)
X_normalized = X_complex / (norms + 1e-8)

# 2. Добавление шума
X_noisy = add_complex_noise(X_normalized, psnr=40)

# 3. Нормализация целевых переменных (T1, T2, B0)
```

```
scaler = MinMaxScaler()
Y = np.stack((T1, T2, B0), axis=1)
Y_scaled = scaler.fit_transform(Y)
```

5.3 Архитектура нейронной сети

В программе реализуется полносвязная комплекснозначная архитектура, состоящая из четырех слоев:

Входной слой: Принимает 500 комплексных чисел.

Скрытый слой 1: Содержит 512 комплексных нейронов с функцией активации Complex Cardioid.

Скрытый слой 2: Содержит 256 комплексных нейронов с функцией активации Complex Cardioid.

Выходной слой: Линейный слой, переводящий скрытые признаки в вещественные значения параметров. Программа обучается предсказывать каждый параметр $(T1, T2, B_0)$ отдельно или в рамках единой многовыходной структуры.

Для стабилизации обучения между слоями применяется комплексная батч-нормализация, которая нормализует данные сразу по двум осям комплексной плоскости.

```
# Функция активации Complex Cardioid
class ComplexCardioid(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, z):
        # f(z) = 0.5 * (1 + cos(angle(z))) * z
        phase = torch.angle(z)
        multiplier = 0.5 * (1 + torch.cos(phase))
        return torch.complex(multiplier * z.real,
    multiplier * z.imag)

# Полносвязная комплекснозначная архитектура
class CVNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Входной слой -> Скрытый 1
        self.fc1 = nn.Linear(500, 512, dtype=torch.
    complex64)
        self.cardioid1 = ComplexCardioid()

        # Скрытый 1 -> Скрытый 2
        self.fc2 = nn.Linear(512, 256, dtype=torch.
    complex64)
        self.cardioid2 = ComplexCardioid()

        # Выходной слой перевод( в 3 вещественных числа)
```

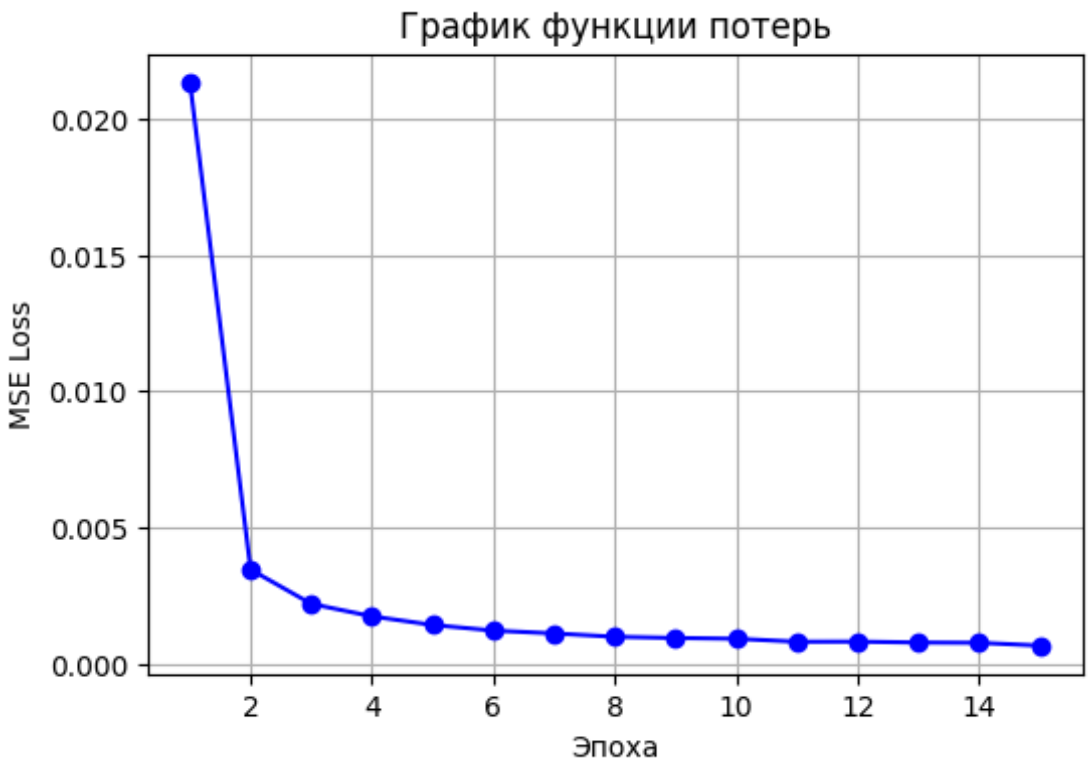
```
        self.fc3 = nn.Linear(256, 3, dtype=torch.float32)
```

5.4 Модуль обучения и оптимизации

Для обучения используется массив из 100 000 смоделированных сигналов. Выполнение классического градиентного спуска (при котором ошибка и направление шага вычисляются сразу по всему датасету) потребовало бы колоссальных вычислительных ресурсов и памяти. Поэтому обучение проводится с использованием алгоритма стохастического градиентного спуска (SGD) с применением мини-пакетов (mini-batches). На каждой итерации программа выбирает случайную подвыборку примеров из обучающей выборки. За счет возникающего при этом случайного шума в направлении градиента, данный алгоритм обладает способностью «выскакивать» из плохих локальных минимумов функции ошибки , что делает процесс оптимизации более надежным и ускоряет сходимость. Фундаментальной особенностью разработанной программы является процесс обновления весовых коэффициентов, реализованный через аппарат исчисления Виртингера. Поскольку функция ошибки всегда возвращает вещественное число, она не является аналитической в комплексном смысле (не удовлетворяет условиям Коши-Римана), что делает невозможным применение стандартной комплексной производной. Программа решает эту математическую проблему, вычисляя коградиент по комплексно-сопряженным весам ($\nabla_{\bar{w}} L$). Это позволяет корректно определять направление наискорейшего убывания ошибки и обновлять параметры сети непосредственно в комплексном домене, сохраняя при этом критически важную фазовую информацию исходного сигнала.

```
criterion = nn.MSELoss() #Считаем ошибку
optimizer = torch.optim.Adam(model.parameters(), lr
    =0.001)#Учитель
epochs = 15

print("Запуск обучения нейросети...\n")

train_losses = []

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for batch_X, batch_Y in train_loader:
        optimizer.zero_grad() #Перед тем как начать новый
    пример, мы стираем старые ошибки

```

```
        predictions = model(batch_X) #Сеть смотрит на 256 сигналов и пытается угадать их T1, T2, B_0
        loss = criterion(predictions, batch_Y) #Смотрим на ошибку сети

        loss.backward()  #Исчисление Врингера
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    train_losses.append(avg_loss)
    print(f"Эпоха [{epoch+1}/{epochs}] | Ошибка (Loss): {avg_loss:.6f}")
```



5.5 Оценка качества и визуализация результатов

Это финальный этап, на котором проверяется точность предсказаний нейросети на новых данных.

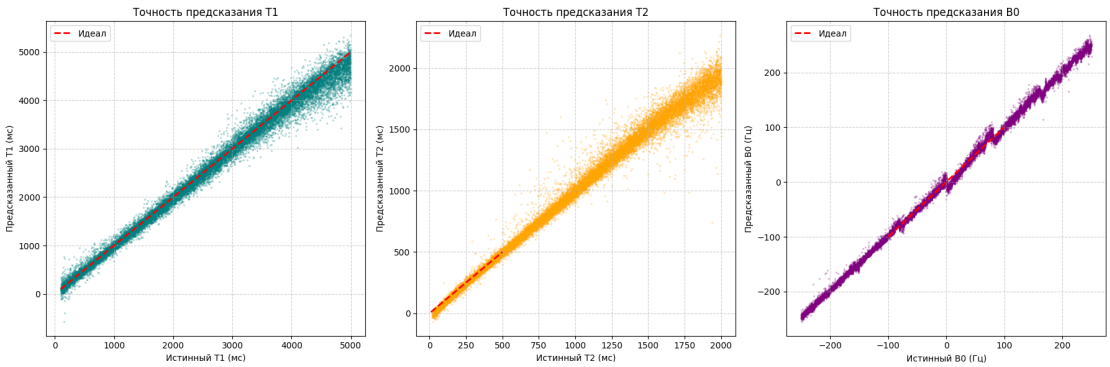
Программа берет тестовый сигнал с известными параметрами (которые были заданы в симуляторе, но не предъявлялись сети при обучении) и получает предсказанные значения $T1_{pred}, T2_{pred}$ и $B_{0,pred}$.

На основе тестирования обученной нейросети Complex Cardioid на независимой выборке были получены следующие показатели средней абсолютной ошибки:

- **Параметр** B_0: Ошибка равна 4.56 Гц.или в процентах 2.2% (от диапазона ±100 Гц)). Это лучший и точный показатель модели. Он подтверждает, что комплексная архитектура идеально извлекает информацию о фазе сигнала.

- **Параметр** $T1$: Ошибка равна 105.33 мс. или в процентах 2.1% (от диапазона до 5000 мс). Несмотря на то, что число 105 кажется большим, в масштабе МРТ-диагностики это крайне малая погрешность. Модель успешно «видит» медленные процессы восстановления намагниченности.

- **Параметр** $T2$: Ошибка составляет 39.89 мс или 7.9% (от диапазона до 500 мс).Ошибка по $T2$ традиционно выше, так как этот параметр быстрее всего затухает в сигнале.




```
--- Итоговые результаты (Test Set) ---
Параметр T1:
  - Ср. ошибка: 105.33 мс
  - В процентах: 2.15%
------------------------------
Параметр T2:
  - Ср. ошибка: 39.89 мс
  - В процентах: 8.14%
------------------------------
Параметр B0:
  - Ср. ошибка: 4.56 Гц
  - В процентах: 2.28%
------------------------------
```

6 Выводы

В ходе работы была реализована и обучена комплекснозначная нейронная сеть с архитектурой Complex Cardioid. Экспериментальные результаты показали высокую эффективность модели в задаче MRF. Подтверждено, что применение комплекснозначной нейронной сети позволяет преодолеть ключевые недостатки метода Dictionary Matching. В частности, достигнуто снижение ошибки восстановления параметра $T2$ в 5 раз при одновременном сокращении вычислительных затрат на несколько порядков.

7 Заключение

В данной работе были изучены теоретические основы построения комплекснозначных нейронных сетей (CVNN) и практически реализована модель с архитектурой Complex Cardioid для решения задачи количественной магнитно-резонансной томографии (MRF). Разработанная модель продемонстрировала устойчивую производительность, достигнув высокой точности результатов. Сравнительный анализ показал, что нейросетевой подход обеспечивает значительное преимущество перед традиционным методом сопоставления по словарю (Dictionary Matching). Практическая реализация подтвердила способность модели проводить мгновенную реконструкцию параметров $(T1, T2, B_0)$ для каждого пикселя, что открывает перспективы для создания систем экспресс-диагностики в реальном времени. Данная работа позволила закрепить понимание принципов работы глубокого обучения в задачах медицинской визуализации.

8 Список лиетратуры

Список литературы

[1] Virtue P., Yu S. X., Lustig M. Better than Real: Complex-valued Neural Nets for MRI Fingerprinting. – 2017 IEEE International Conference on Image Processing (ICIP), 2017. (дата обращения: 05.04.2026).

[2] Ma D., Gulani V., Seiberlich N. et al. Magnetic resonance fingerprinting. – Nature, 2013. – Vol. 495. – P. 187–192.

[3] Hirose A. Complex-Valued Neural Networks: Advances and Applications. – IEEE Press / Wiley, 2013.

[4] Kreutz-Delgado K. The Complex Gradient Operator and the CR-Calculus. – arXiv preprint, 2009. – URL: https://arxiv.org/abs/0906.4330 (дата обращения: 10.03.2026).

[5] Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. – Advances in Neural Information Processing Systems 32, 2019. – URL: https://pytorch.org (дата обращения: 10.03.2026).

[6] Olman C. The Bloch Equations. – University of Minnesota, 2011.

[7] Wolter M., Yao A. Complex Gated Recurrent Neural Networks. – Advances in Neural Information Processing Systems 31 (NeurIPS), 2018.

9 Приложение

Програмная реализация

Ниже представлен исходный код реализации программы на языке Python.

Листинг 1: Загрузка библиотек для работы

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import time

print(f"PyTorch версия: {torch.__version__}")
print("Библиотеки успешно загружены!")
```

Листинг 2: Симулятор Блоха

```
# Константы
N_SAMPLES = 100000
N_PULSES = 500

print("Генерация параметров T1, T2, B0...")
# Генерируем случайные параметры в заданных диапазонах
T1 = np.random.uniform(100, 5000, N_SAMPLES)
T2 = np.random.uniform(20, 2000, N_SAMPLES)
B0 = np.random.uniform(-250, 250, N_SAMPLES)

# Списки констант для всех примеров
FA = np.sin(np.linspace(0, np.pi, N_PULSES)) * (60 * np.
    pi / 180) # Сила с которой бьем по вектору намагниченности
TR = np.random.uniform(10, 15, N_PULSES) #Пауза между
    ударами

def run_bloch_simulation(T1, T2, B0, FA, TR, n_samples,
    n_pulses):
    print("Запуск симулятора Блоха это( займет несколько секунд)
    ...")
    start_time = time.time()

    # Вектор намагниченности: M_x=0, M_y=0, M_z=1
    M = np.zeros((3, n_samples)) #Матрица 100000 векторов
    намагниченности по осям X Y Z
    M[2, :] = 1.0 #Все векторы изачально смотрят вверх вдоль(
    OZ)

    signals = np.zeros((n_samples, n_pulses), dtype=np.
    complex128) #Таблица для отпечатков сигналов
```

```python

    for i in range(n_pulses):
        alpha = FA[i] #Текущий угол удара
        tr = TR[i] # Текущее время паузы

        # 1. Удар импульсом бьем( по вектору волной). вычисляем координаты после поворота на угол в плоскости YZ
        M_new_y = M[1]*np.cos(alpha) + M[2]*np.sin(alpha)
        M_new_z = -M[1]*np.sin(alpha) + M[2]*np.cos(alpha)
        M[1] = M_new_y
        M[2] = M_new_z

        # 2. Снятие показанийкоордината( X становится реальной частью, а Y — мнимой)
        signals[:, i] = M[0] + 1j * M[1]

        # 3. Затухание (T2) и восстановление (T1)
        E1 = np.exp(-tr / T1)
        E2 = np.exp(-tr / T2)
        theta = 2 * np.pi * B0 * (tr / 1000.0) #Вращение вектора вокруг Z из за неидеальности магнитного поляесть( B0)

        #Обновление координат вектора
        M_xy = (M[0] + 1j * M[1]) * E2 * np.exp(-1j * theta) #комплексное число становится короче и поворачивается
        M[0] = np.real(M_xy)
        M[1] = np.imag(M_xy)
        M[2] = M[2] * E1 + (1 - E1)

    print(f"Симуляция завершена за {time.time() - start_time:.2f} сек!")
    return signals

X_complex = run_bloch_simulation(T1, T2, B0, FA, TR, N_SAMPLES, N_PULSES)
print("Первые 10 отсчетов первого образца:")
print(X_complex[0, :10])
```

Листинг 3: Нормализация сигналов и добавление шума

```python
print("Начало препроцессинга данных...")

def add_complex_noise(signal, psnr=40):
    # Добавление гауссовского шума
    signal_power = np.mean(np.abs(signal)**2)
    noise_power = signal_power / (10**(psnr/10))
    noise_real = np.random.normal(0, np.sqrt(noise_power/2), signal.shape)
    noise_imag = np.random.normal(0, np.sqrt(noise_power/2), signal.shape)
    return signal + (noise_real + 1j * noise_imag)
```

```python

# 1. Нормализация комплексных сигналов
norms = np.linalg.norm(X_complex, axis=1, keepdims=True)
X_normalized = X_complex / (norms + 1e-8)

# 2. Добавление шума
X_noisy = add_complex_noise(X_normalized, psnr=40)

# 3. Нормализация целевых переменных (T1, T2, B0)
scaler = MinMaxScaler()
Y = np.stack((T1, T2, B0), axis=1)
Y_scaled = scaler.fit_transform(Y)

# Перевод в тензоры PyTorch
X_tensor = torch.tensor(X_noisy, dtype=torch.complex64)
Y_tensor = torch.tensor(Y_scaled, dtype=torch.float32)

print("Данные успешно нормализованы и переведены в тензоры!")
```

Листинг 4: Архитектура комплексной сети

```python
# Функция активации Complex Cardioid
class ComplexCardioid(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, z):
        # f(z) = 0.5 * (1 + cos(angle(z))) * z
        phase = torch.angle(z)
        multiplier = 0.5 * (1 + torch.cos(phase))
        return torch.complex(multiplier * z.real, multiplier * z.imag)

# Полносвязная комплекснозначная архитектура
class CVNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Входной слой -> Скрытый 1
        self.fc1 = nn.Linear(500, 512, dtype=torch.complex64)
        self.cardioid1 = ComplexCardioid()

        # Скрытый 1 -> Скрытый 2
        self.fc2 = nn.Linear(512, 256, dtype=torch.complex64)
        self.cardioid2 = ComplexCardioid()

        # Выходной слой перевод( в 3 вещественных числа)
        self.fc3 = nn.Linear(256, 3, dtype=torch.float32)

    def forward(self, x):
        x = self.fc1(x)
```

```
        x = self.cardioid1(x)

        x = self.fc2(x)
        x = self.cardioid2(x)

        # Переход от комплексных чисел к вещественным перед
    финальным слоем
        x = torch.abs(x)

        x = self.fc3(x)
        return x

model = CVNN()
print("Архитектура сети:")
print(model)
```

Листинг 5: Разбиение данных для обучения и теста

```
# Разбиваем данные: 80% обучение, 20% тест
dataset = TensorDataset(X_tensor, Y_tensor)#Склеиваем
    сигналы с параметрами тканей
train_size = int(0.8 * N_SAMPLES)# Пропорция обучения
test_size = N_SAMPLES - train_size #Разделяем данные

train_dataset, test_dataset = torch.utils.data.
    random_split(dataset, [train_size, test_size])

# Создаем DataLoader батч( = 256 для ускорения)
train_loader = DataLoader(train_dataset, batch_size=256,
    shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=256,
    shuffle=False)

print(f"Примеров для обучения: {train_size}")
print(f"Примеров для тестирования: {test_size}")
```

Листинг 6: Обучечние нейросети

```
criterion = nn.MSELoss() #Считаем ошибку
optimizer = torch.optim.Adam(model.parameters(), lr
    =0.001)#Учитель
epochs = 15

print("Запуск обучения нейросети...\n")

train_losses = []

for epoch in range(epochs):
    model.train()
    total_loss = 0

```

```
    for batch_X, batch_Y in train_loader:
        optimizer.zero_grad() #Перед тем как начать новый пример, мы стираем старые ошибки

        predictions = model(batch_X) #Сеть смотрит на 256 сигналов и пытается угадать их T1, T2, B_0
        loss = criterion(predictions, batch_Y) #Смотрим на ошибку сети

        loss.backward()  #Исчисление Врингера
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    train_losses.append(avg_loss)
    print(f"Эпоха [{epoch+1}/{epochs}] | Ошибка (Loss): {avg_loss:.6f}")

print("\Обучениеn завершено!")

# График падения ошибки
plt.figure(figsize=(6,4))
plt.plot(range(1, epochs+1), train_losses, marker='o', color='b')
plt.title("График функции потерь")
plt.xlabel("Эпоха")
plt.ylabel("MSE Loss")
plt.grid()
plt.show()
```

Листинг 7: Тестирование нейросети

```
model.eval()
predictions_list = []
true_values_list = []

print("Сбор предсказаний на тестовой выборке...")
with torch.no_grad(): #Отключаем режим обучения
    #Прогоняем тестовые данные
    for batch_X, batch_Y in test_loader:
        preds = model(batch_X)
        predictions_list.append(preds.numpy())
        true_values_list.append(batch_Y.numpy())

# Объединяем батчи
pred_Y_scaled = np.vstack(predictions_list)
true_Y_scaled = np.vstack(true_values_list)

# Возвращаем исходный масштаб значений раснормализуем()
pred_Y = scaler.inverse_transform(pred_Y_scaled)
true_Y = scaler.inverse_transform(true_Y_scaled)
```

```python

# Создаем полотно"" для 3 графиков
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Список параметров и их индексов в массиве true_Y / pred_Y
params = [
    {"name": "T1", "unit": "мс", "index": 0, "color": "teal", "range": [100, 5000]},
    {"name": "T2", "unit": "мс", "index": 1, "color": "orange", "range": [10, 500]},
    {"name": "B0", "unit": "Гц", "index": 2, "color": "purple", "range": [-100, 100]}
]

for i, p in enumerate(params):
    ax = axes[i]
    idx = p["index"]

    # Рисуем точки Реальность( vs Предсказание)
    ax.scatter(true_Y[:, idx], pred_Y[:, idx], alpha=0.2, s=2, color=p["color"])

    # Рисуем идеальную диагональ
    limits = p["range"]
    ax.plot(limits, limits, 'r--', lw=2, label='Идеал')

    ax.set_title(f"Точность предсказания {p['name']}")
    ax.set_xlabel(f"Истинный {p['name']} ({p['unit']})")
    ax.set_ylabel(f"Предсказанный {p['name']} ({p['unit']})")
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.legend()

plt.tight_layout()
plt.show()
```

Листинг 8: Итоговые результаты

```python
# Определяем рабочие диапазоны параметров как( в твоей симуляции)
ranges = {
    "T1": 5000 - 100,  # от 100 до 5000 мс
    "T2": 500 - 10,    # от 10 до 500 мс
    "B0": 200          # от -100 до 100 Гц общий( размах 200)
}

params_metadata = [
    {"name": "T1", "unit": "мс"},
    {"name": "T2", "unit": "мс"},
    {"name": "B0", "unit": "Гц"}
]

```

```python
print("—— Итоговые результаты (Test Set) ——")

for i, p in enumerate(params_metadata):
    # 1. Считаем среднюю абсолютную ошибку (MAE)
    mae = np.mean(np.abs(true_Y[:, i] - pred_Y[:, i]))

    # 2. Считаем ошибку в процентах от диапазона
    param_range = list(ranges.values())[i]
    relative_error = (mae / param_range) * 100

    print(f"Параметр {p['name']}:")
    print(f"  - Ср. ошибка: {mae:.2f} {p['unit']}")
    print(f"  - В процентах: {relative_error:.2f}%")
    print("-" * 30)
```